BIT 326 DISTRIBUTED SYSTEMS
Chapter 8 – Data Management & Distributed Transactions in Distributed Systems

8.1 Introduction

Data management in distributed systems involves coordinating, storing, and processing data across multiple, geographically dispersed nodes to ensure high availability, scalability, and performance. The fundamental principle behind data management is data independence, which enables applications and users to share data at a high conceptual level while ignoring implementation details. This principle has been achieved by database systems which provide advanced capabilities such as schema management, high-level query languages, access control, automatic query processing and optimization, transactions, data structures for supporting complex objects, etc.

Distributed transactions are sets of operations involving multiple, distinct networked systems or databases that must all succeed or all fail together (atomicity) to ensure data consistency, commonly managed through protocols like two-phase commit (2PC) or Saga patterns.

Distributed data management (DDM) in distributed systems involves handling data spread across multiple locations, making it appear as a single system for users, focusing on scalability, availability, and consistency through techniques like replication and partitioning, and requiring careful handling of challenges like consistency and security to support complex modern applications.

Key techniques include data replication for fault tolerance, sharding for horizontal scaling, and consistency protocols to maintain data integrity. It enables users to access remote, distributed data seamlessly as if it were local.

8.2 Distributed Data

Distributed data refers to a data management approach where data is stored across multiple locations or nodes instead of being centralized in a single location. This method improves data access speed, fault tolerance, and scalability, making it essential in today's applications. Key concepts like data partitioning, replication, and consistency models play a crucial role in distributed data systems

8.3. Importance of Distributed Data Architecture

The importance of distributed data architecture can be highlighted through several key points:

Key Benefits	Description
Speed and Performance	Distributed systems can process data faster by utilizing multiple nodes, allowing for parallel processing and reduced latency.
Fault Tolerance	By replicating data across different nodes, distributed systems can remain operational even if one or more nodes fail, ensuring high availability.
Scalability	Distributed data architectures can easily scale horizontally by adding more nodes to the system, accommodating growing data volumes and user demands.
Flexibility	These systems can support various data types and structures, making them suitable for diverse applications.
Geographical Distribution	Data can be stored closer to where it is needed, reducing access times for users across different regions.

8.4. Key Components of Distributed Data Architecture

In a distributed data architecture, several key components work together to ensure efficient data management:

1) Nodes: Individual servers or data storage units that hold portions of the overall dataset.
2) Clusters: Groups of nodes that work together to provide a unified data service.
3) Data Shards: Segments of data that are distributed across nodes for load balancing and performance optimization.
4) Data Replication: The process of copying data across multiple nodes to ensure durability and availability.
   - Data Replication is a critical mechanism in distributed systems, ensuring that copies of data exist on multiple nodes. This redundancy enhances data durability and availability. For example, using a replication factor of three means that each piece of data is stored on three different nodes, providing a safeguard against data loss.
5) Consistency Models: Different strategies for ensuring that all nodes reflect the same data state, including strong consistency, eventual consistency, and causal consistency

8.5. Tools and Technologies for Distributed Data

Several tools and technologies facilitate the implementation and management of distributed data systems:

1) Distributed Databases
   a) Apache Cassandra: A highly scalable NoSQL database designed to handle large amounts of data across many commodity servers.
   b) MongoDB: A document-oriented NoSQL database that allows for flexible data modeling and efficient querying.
   c) Amazon DynamoDB: A fully managed NoSQL database service that provides fast and predictable performance with seamless scalability.
2) Data Streaming Platforms
   - Apache Kafka: A distributed streaming platform that enables real-time data processing and analytics.
3) Cloud-Based Platforms
   - Utilizing cloud platforms like AWS, Google Cloud, and Microsoft Azure can simplify the deployment of distributed data systems, providing scalability and reliability.
4) Containerization and Orchestration Tools
   - Containerization packages applications with dependencies into portable units, while orchestration tools manage, scale, and automate these containers across distributed systems. Key tools include Docker for packaging and Kubernetes for managing complex, containerized workloads. These technologies ensure consistent deployment, high availability, and efficient resource utilization. They enhance the consistency and manageability of distributed data environments, allowing for automation in deployment and scaling.
5) Chat2DB: A Comprehensive Solution
   - Chat2DB stands out as an AI-powered database management tool designed to simplify the complexities of distributed data systems. By leveraging AI technology, Chat2DB enhances database management efficiency, providing features like natural language SQL generation, intelligent SQL editing, and automated data analysis

8.5.1. Distributed Database

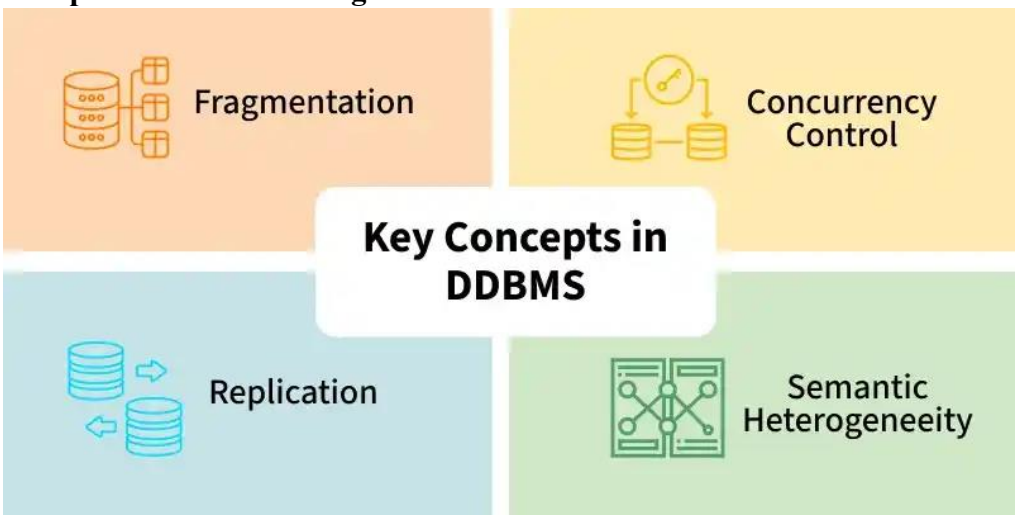
A distributed database is a system where data is stored across multiple, physically separated nodes (servers) in the same or different locations, appearing as a single, unified database to users. It enhances performance, scalability, and high availability by using techniques like data sharding (partitioning) and replication. The database is scattered over various locations which provides local access to data and thus reduces communication costs and increases availability.

Distributed Database Systems classification

Broadly classified into two types.

1) Homogeneous Distributed System - In Homogenous distributed database system, the data is distributed but all servers run the same Database Management System (DBMS) software.
2) Heterogeneous Distributed System–In Heterogeneous distributed databases different sites run under the control of different DBMSs. These databases are connected somehow to enable access to data from multiple sites.

Components and Challenges



1) **Fragmentation:** Fragmentation divides a database into smaller, logically related units called fragments.
   - **Horizontal Fragmentation:** Splits tables by rows (tuples) based on specific conditions, such as grouping customer data by region.
   - **Vertical Fragmentation:** Splits tables by columns (attributes), linking them via primary keys.
   - **Hybrid Fragmentation:** A combination of horizontal and vertical techniques.
   - **Purpose:** Improves query performance by keeping data closer to where it is used and increases security.
2) Replication: Replication involves maintaining multiple copies of the same data at different sites to increase reliability and access speed.
   - **Full Replication:** The entire database is copied to every site.
   - **Partial Replication:** Only frequently used data is replicated at specific sites.
   - **No Replication:** Each fragment is stored at only one location.
   - **Purpose:** Improves data availability and fault tolerance (if one site fails, others are available).

Data fragmentation and replication are distributed database design techniques used to improve performance, reliability, and data access speed.

3) **Concurrency Control:** Concurrency control ensures data remains accurate when multiple transactions run at the same time. Without it, issues like lost updates or dirty reads can occur. Its goal is to make parallel transactions behave as if run one by one. Common methods include locking, timestamps, and optimistic concurrency.
4) **Semantic Heterogeneity: Semantic heterogeneity** happens when different databases use the same data labels but with different meanings, formats, or units. For example, one system may store salary in dollars, another in rupees. This can cause confusion during data integration, so resolving it is important for accurate results.

8.5.2. Data Streaming Platforms

Data streaming platforms (DSP) in distributed systems are software infrastructures designed to ingest, process, and analyze continuous, high-velocity data flows in real-time or near real-time. A data streaming platform (DSP) enables organizations to capture, store, and process data as a continuous flow of real time events. By leveraging event-driven architectures powered by foundational technologies like Apache Kafka®, Apache Flink® and Amazon Kinesis a data streaming platform allows businesses to unlock real-time insights, build responsive applications, and connect diverse systems with unprecedented agility. With integrated tools for governance, schema management, and self-service access, a data streaming platform simplifies complex data architectures and enables teams to deliver real-time value from data streams

Key Components of Data Streaming Platforms

Modern data streaming platforms generally consist of six core components to manage the lifecycle of streaming data:

1) **Real-Time Data Streaming (Ingestion/Messaging):** Technologies like Apache Kafka or Redpanda serve as the backbone, providing scalable, durable, and fault-tolerant storage for event logs, allowing producers and consumers to be decoupled.
2) **Stream Processors:** Engines such as Apache Flink, Apache Spark Streaming, or Kafka Streams transform, enrich, and aggregate data in-flight.
3) **Schema Registry:** Ensures data consistency and compatibility by managing schema evolution (e.g., Avro, JSON, Protobuf), allowing producers and consumers to evolve without breaking pipelines.
4) **Connectors (Kafka Connect):** Pre-built connectors (e.g., CDC, JDBC) allow seamless ingestion and delivery of data between the streaming platform and external systems like databases, SaaS apps, or cloud storage.
5) **Data Portal/Management:** Provides self-service interfaces for discovering, managing, and tracking the lineage of data streams.
6) **Tableflow/Table Storage:** Tools that materialize stream data into tables (like Apache Iceberg) to bridge the gap between real-time and historical analytics

Key Benefits

1) Reduced Latency: Enables real-time insights, reducing response times from hours to milliseconds, which is critical for fraud detection and user experience.
2) Decoupled Architecture: Breaks dependencies between microservices, improving system reliability and allowing teams to scale independently.

3) Scalability & Durability: Distributed architectures allow handling massive, surging data volumes while maintaining data safety.
4) Real-time Analytics: Enables continuous processing, such as updating machine learning models or live dashboards.

8.5.3. Cloud Platforms

Cloud platforms are internet-based services offering on-demand IT infrastructure, including computing, storage, networking, and application development tools (IaaS, PaaS, SaaS), removing the need for companies to maintain physical hardware. These platforms enhance performance, reduce latency by dispersing resources geographically, and ensure high availability for global applications, essential for modern microservices and edge computing.

Architecture & Components: These systems use virtualization to create virtual instances (VMs) on physical servers, utilizing technologies like hypervisors (e.g., KVM, VMware) to manage resources.

Key Benefits:

1) Scalability & Elasticity: Resources can be automatically added or reduced, supporting rapid growth and demand fluctuations.
2) Low Latency & Performance: By placing compute resources closer to users, distributed clouds reduce latency and enhance performance, which is crucial for IoT and real-time applications.
3) Reliability & Fault Tolerance: If one node or server fails, the system continues to operate, often by replicating data across multiple data centers.

Deployment Models:

1) **Public Clouds:** Third-party providers (e.g., AWS, Google Cloud) offer services via the internet.
2) **Private/Hybrid Clouds:** Private infrastructure or a mix of private and public, often offering more tailored security.

Example

1) Amazon Web Services**:** The largest, most comprehensive provider offering 200+ services, focusing on scalable compute and storage.
2) Microsoft Azure**:** A top competitor, highly favoured by enterprises for integration with Windows and other Microsoft tools
3) Google Cloud Platform (GCP): Recognized for strong data analytics, machine learning, and AI capabilities.
4) Alibaba Cloud Leading provider in China and Asia-Pacific, offering strong IaaS for retail and finance.
5) Oracle Cloud: Strong in ERP, databases, and enterprise applications with a focus on hybrid cloud.
6) IBM Cloud: Focused on hybrid cloud, AI, and blockchain services

8.5.4. Containerization and Orchestration Tools

Containerization packages applications with dependencies into portable units (e.g., Docker, Podman), ensuring consistent execution across distributed environments. Orchestration tools (e.g., Kubernetes, Docker Swarm) automate the lifecycle, scaling, and deployment of these containers across server clusters. This combination maximizes efficiency, self-healing, and resource utilization in modern cloud-native architectures.

Containerization and orchestration tools are essential for managing distributed systems, automating the deployment, scaling, and operational lifecycle of applications across multiple servers.

Benefits in Distributed Systems

1) **Automation:** Automates deployment, networking, and scaling.
2) **Self-Healing:** Automatically restarts failed containers and schedules replacements.
3) **Resource Optimization:** Efficiently balances workload demands across nodes to save costs.
4) **Consistency:** Eliminates environment discrepancies between development and production
5) **Scaling:** Instantly increases or decreases container instances based on demand.
6) **Load Balancing:** Distributes network traffic to ensure stability

8.6. Distributed Data Management Techniques & Strategies

1) Data Replication**:** Storing copies of data on multiple nodes ensures high availability and improves fault tolerance.
2) Sharding (Partitioning)**:** Breaking large datasets into smaller, manageable pieces spread across different nodes to improve performance.
3) Data Fragmentation: Dividing data horizontally (rows) or vertically (columns) to optimize access.
4) Consistency Models**:** Using protocols (like Paxos or Raft) to manage trade-offs between consistency and availability (CAP theorem), ensuring either immediate or eventual consistency across nodes.
5) Data Locality**:** Placing data close to the applications that use it most to reduce network latency.

8.7. Best Practices for Managing Distributed Data

Managing distributed data systems effectively involves following several best practices:

1) Choosing the Right Consistency Model: Selecting an appropriate consistency model based on application requirements is crucial. For instance, real-time applications may benefit from strong consistency, while social media platforms might opt for eventual consistency to enhance performance.
2) Effective Data Partitioning Effective Data Partitioning: Utilizing hash functions or range-based partitioning can optimize data distribution and retrieval. For example, a hash function can be applied to user IDs to determine where user data should be stored across nodes.

```
CREATE TABLE users (
user_id INT PRIMARY KEY,
username VARCHAR(100),
email VARCHAR(100)
) PARTITION BY HASH(user_id);
```

3) Monitoring and Logging: Implementing comprehensive monitoring and logging practices can help identify performance issues and ensure system health. Tools like Prometheus and Grafana can provide insights into system performance metrics.
4) Regular Backups and Data Recovery Plans: Establishing regular backup routines and robust data recovery plans is essential to mitigate data loss risks. Automated backup solutions can help streamline this process.
5) Security Protocols: Implementing strong security protocols is vital to protect data across distributed nodes. Techniques like encryption, access controls, and secure communication channels should be employed.

6) Containerization and Orchestration Tools: Using containerization tools like Docker in conjunction with orchestration platforms such as Kubernetes can simplify the management of distributed data systems. These tools enable consistent deployment and scalability.

8.8. Challenges in Distributed Data Management:

While distributed data systems offer numerous advantages, they also present several challenges:

1) Ensuring Data Consistency: Maintaining data consistency across geographically dispersed nodes can be complex. Conflicts may arise due to network delays or concurrent updates.
2) Network Latency: Network latency can significantly impact data access speed and overall system performance. Techniques such as caching and content delivery networks can help mitigate these effects.
3) Fault Tolerance: Achieving fault tolerance requires redundancy and careful planning. Systems must be designed to quickly recover from node failures without losing data.
4) Data Skew and Hot Spots: Data skew occurs when certain nodes handle disproportionately high loads, leading to performance bottlenecks. Monitoring and load balancing strategies can help address these issues.
5) Data Privacy and Security: Ensuring data privacy and security in distributed systems is paramount. Implementing security measures across all nodes is essential to protect sensitive information.
6) Scalability & Load balancing Challenges: As data volumes and user demands grow, scaling distributed data systems can become challenging. Strategies such as sharding and load balancing are necessary to accommodate increasing demands.
7) Concurrency control: Coordinating concurrent access to data across nodes to prevent race conditions and ensure data integrity.
8) Replication Management: Deciding how to distribute and synchronize data copies (replicates) across nodes.
9) Security: Protecting data across multiple distributed locations from unauthorized access.
10) Heterogeneity: Managing data across varying hardware, software, and network standards.

8.9. Future Trends in Data Management in Distributed Data Systems

As the field of distributed data systems evolves, several emerging trends are shaping its future:

1) **Rise of Edge Computing:** Edge computing is gaining traction as data processing occurs closer to data sources, reducing latency and improving real-time analytics.
2) **AI and Machine Learning Integration:** The integration of AI and machine learning techniques will drive optimizations in distributed data management, enabling smarter decision-making and predictive analytics.
3) **Blockchain Technology:** Blockchain technology offers potential enhancements to data security and integrity in distributed systems, making it a compelling area of exploration.
4) **Serverless Computing:** Serverless architectures simplify the deployment and scaling of distributed applications, reducing the operational overhead for developers.
5) **Energy Efficiency and Sustainability:** As environmental concerns grow, there is an increasing focus on energy-efficient data centers and sustainable practices in distributed data management.
6) **Quantum Computing:** Quantum computing has the potential to revolutionize data processing and analysis in distributed environments, offering unprecedented speed and efficiency.

8.11. Distributed Transactions in Distributed Systems

In the world of distributed computing and distributed transactions,

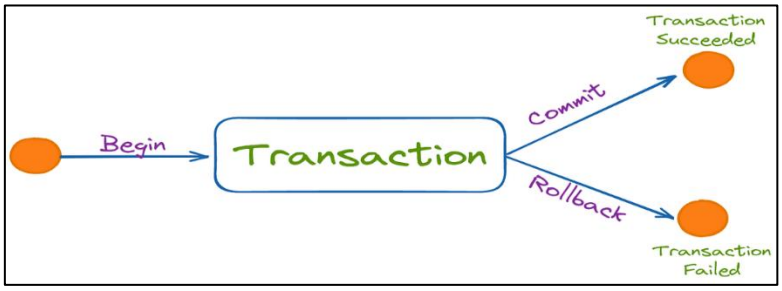
- a transaction can be defined as a group of statements representing a unit of work composed of a set of operations on objects.
- A transactions can also be viewed as sequences of operations on resources, such as read, write, or update, which transform one consistent state of the system into a new consistent state.
- A transaction is a collection of operation that represents a unit of consistency and recovery
- A transaction is a unit of work that ensures data consistency — either all operations succeed (commit) or none take effect (rollback).

What is a Distributed Transaction?

1) A distributed transaction is a set of operations on data that is performed across two or more data repositories (especially databases).
2) A distributed transaction is a complex operation in computer science and database management, where a single transaction involves multiple interconnected systems or databases.
3) Distributed transaction processing often involves a series of sub-operations, and they must either succeed in their entirety or fail completely, adhering to the principles of Atomicity, Consistency, Isolation, and Durability (ACID).

A distributed transaction spans multiple systems, ensuring all operations either succeed or fail together, crucial for maintaining data integrity and consistency across diverse and geographically separated resources in modern computing environments. There are two possible outcomes:

- all operations successfully complete,
- none of the operations are performed at all due to a failure somewhere in the system.




```
BEGIN TRANSACTION;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

These transactions are crucial in distributed computing environments, ensuring data integrity and consistency across the network while allowing multiple systems to work together seamlessly.

8.12 Key Characteristics of Distributed Transactions:

1) ACID Compliance: Like traditional transactions, they must adhere to Atomicity (all or nothing), Consistency (valid state transitions), Isolation (independent execution), and Durability (permanent results).

2) Multi-Resource Participation: The transaction spans multiple, distinct databases, microservices, or systems connected over a network.
3) Centralized or Distributed Coordination: A transaction manager (or coordinator) must manage the commit or rollback process across all participating nodes.
4) Atomic Commitment Protocol (e.g., 2PC): Often uses protocols like Two-Phase Commit (2PC) or XA transactions to ensure that all nodes reach a consensus, either committing or rolling back as a unit.
5) Network Dependency: Due to the reliance on communication between distributed nodes, they are sensitive to network latency, partitioning, and potential timeouts.
6) Fault Tolerance: The transaction mechanism must handle failures of individual nodes or network failures without breaking the consistency of the entire system.

8.13. Goal of Distributed Transaction

The need for distributed transactions arises from the requirements to ensure data consistence and reliability across multiple independent systems or resources in a distributed computing environment. The goal is;

1) **Atomic Commitment:** Ensuring that even if a transaction spans multiple nodes or services, it is treated as a single, indivisible unit.
   Ensuring that either all operations within a transaction are completed successfully or none of them are, avoiding partial updates that could lead to inconsistencies. A transaction should be treated as a single unit of operation. Only completed transactions are committed and incomplete transactions are rolled back or restored to their original state.
2) **Data Consistency (Validity):** Ensuring that all changes made as part of a transaction are committed or rolled back atomically, maintaining data integrity. Consistency is concerned with correctly reflecting the reality of the state of the resources.
3) **Preventing Partial Updates:** If one participant in the transaction fails, all other participants roll back to the original state to avoid data corruption.
4) **Reliability & Durability:** Ensuring that once a distributed transaction is committed, it remains consistent and persistent, even in the event of system failures.
5) **Reliable Coordination:** Managing transactions efficiently across heterogeneous environments
6) **Isolation (Independent Operation):** Guaranteeing that concurrent transactions do not interfere with each other, preserving data integrity and preventing conflicts.
7) **Durability (Permanence):** Confirming that committed transactions persist even in the event of system failures, ensuring reliability.

8.13. Components of a Distributed Transaction

The key components of a distributed transaction solution encompass a set of crucial elements that collectively enable the seamless execution of transactions across distributed systems. These components work in tandem to facilitate the reliable execution of distributed transactions while adhering to the ACID properties, maintaining data integrity and consistency. These components include:

1) **Participants:** These are the individual systems or databases involved in the transaction, each performing its specific role.
2) **Transaction Manager (Coordinator):** The central entity that manages the transaction lifecycle, coordinating with resource managers to ensure atomicity (all-or-nothing). It is

responsible for orchestrating the transaction. It initiates and oversees the process, ensuring all participants follow the protocol. Utilizes the 2PC protocol to decide whether to commit or abort.

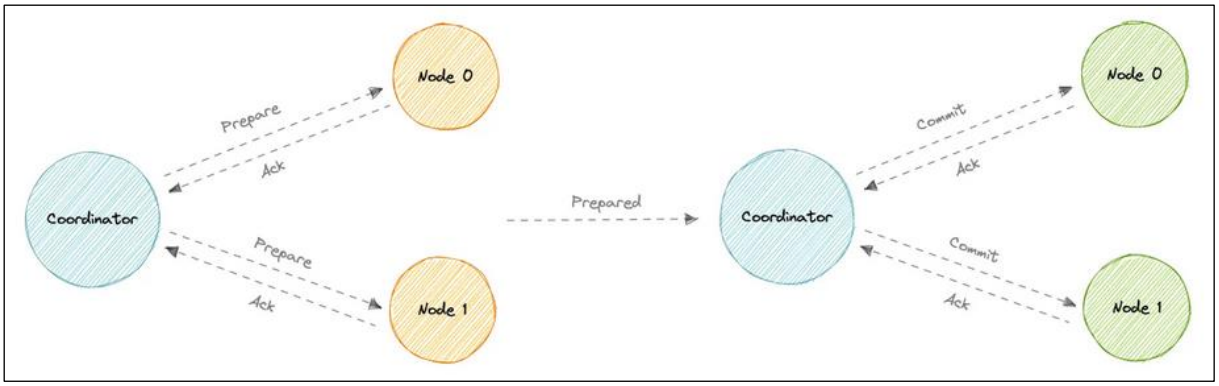
3) **Transaction Identifier (TID):** A unique identifier assigned by the transaction manager to every distributed transaction, allowing all participants to track, identify, and associate actions with that specific transaction.
4) **Transaction Log:** A record of all actions and changes made during the transaction. A durable log, maintained by both the coordinator and participants, that records all transaction actions, enabling recovery of data after system failures. It is essential for rollback or recovery in case of failure.
5) **Resource Manager:** Responsible for managing resources (data or services) for each participant. It handles local operations, and committing or rolling back changes based on instructions from the Transaction Manager. The resource manager participates in distributed transactions by implementing a transaction resource interface. This interface is used by the transaction manager to communicate transaction associations, transaction completion as well as recovery work.
6) **Application Program/Server:** The application that initiates the transaction and communicates with the Resource Managers, often running within an application server container.
7) **Communication Mechanism:** Networks and protocols that facilitate message passing between the coordinator and participants, such as JTA (Java Transaction API) or XA protocol.
8) **Two-Phase Commit (2PC):** A protocol used to achieve consensus among participants, ensuring that either all participants commit or none of them do.

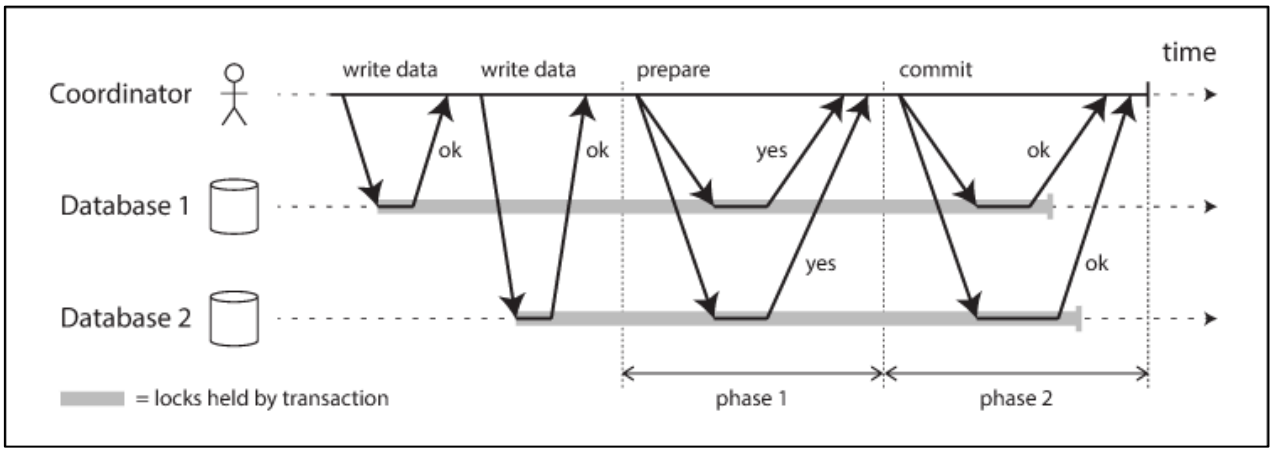
Each of these contributes to the distributed transaction processing system by implementing different sets of transaction APIs and functionalities.

8.14. Protocols & Type of Distributed Transactions

Distributed transactions involve coordinating actions across multiple nodes or resources to ensure atomicity, consistency, isolation, and durability (ACID properties). Here are some common types and protocols:

1) **Two-Phase Commit Protocol (2PC):** This is a classic protocol used to achieve atomicity in distributed transactions. It uses a **coordinator** to orchestrate the transaction across multiple **participants** (databases or services). It involves two phases:
   - a prepare phase where all participants agree to commit or abort the transaction,
   - a commit phase where the decision is executed synchronously across all participants.





2PC ensures that either all involved resources commit the transaction or none do, thereby maintaining atomicity.

Advantages: Strong consistency guarantees, standardised approach.
Limitations: Coordinator failure can block participants – Single point of failure; extra round trips add latency impacting on performance; locks (blocking) held during both phases can reduce throughput.

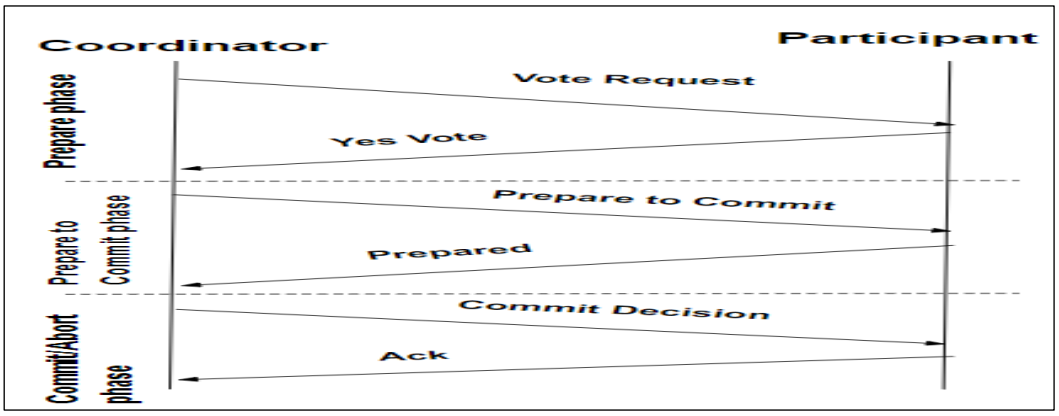
2) **Saga Pattern:** The Saga pattern manages data consistency across distributed microservices by breaking long-running transactions into a sequence of local transactions. Each service performs a local update and triggers the next step; if any step fails, the saga executes compensating transactions to undo the preceding changes.

An alternative for long-running transactions where a sequence of local transactions is executed, with "compensating transactions" (undo operations) triggered if one step fails. It is a design pattern for managing data consistency in distributed systems, such as microservices, by splitting a long-running, distributed transaction into a sequence of smaller local transactions. Each service performs its own local transaction and publishes an event that triggers the next step. If a step fails, the saga executes "compensating transactions" to undo the previous actions, ensuring eventual consistency

Aspect	**2PC**	**Saga**
Consistency	Strong atomic commit across all participants	Eventual, business-level consistency via compensations
Locking	Locks held during prepare to commit phase	Local commit per service, no global locks
Latency	Higher, with blocking risk if coordinator fails	Lower, no blocking, more scalable
Failure mode	Coordinator failure can block, requires recovery	Compensation steps undo failed actions, require idempotency
Example	Payment, inventory, order in single transaction	Steps: order, reserve, pay, with comp. (refund/cancel)

3) In 3PC, **Three-Phase Commit Protocol (3PC):** Non-blocking protocol designed to improve upon 2PC. It adds a "pre-commit" phase to prevent resources from being held indefinitely if the coordinator fails. Participants first agree to prepare to commit, then to commit, and finally to

complete or abort the transaction. This protocol aims to reduce the risk of blocking seen in 2PC by introducing an additional decision-making phase using 'canCommit', 'preCommit', and 'doCommit'



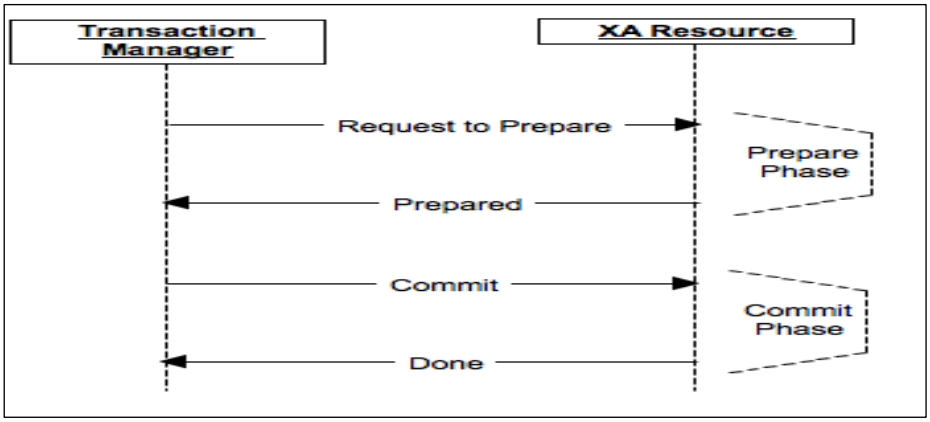
Advantages: Reduces indefinite blocking under specific network conditions, Improved fault tolerance, data consistency and Reduced global deadlock risk

Limitations: Assumes bounded network delays and no partitions; more complex and rarely used in practice compared to 2PC.

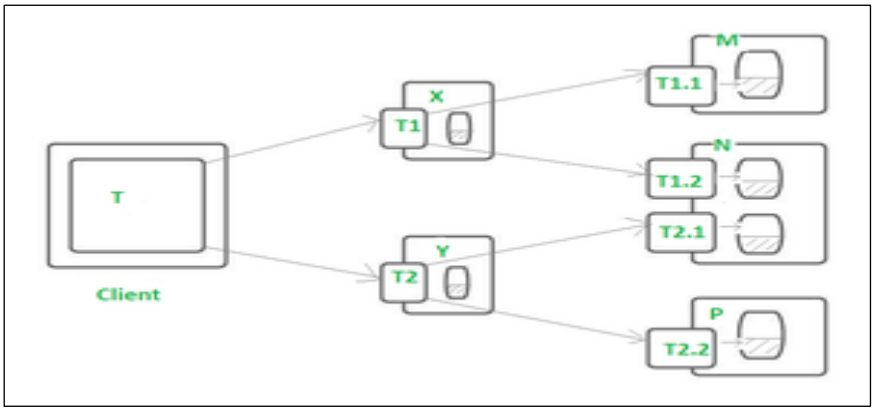
4) **XA Transactions:** XA (eXtended Architecture); Based on the X/Open XA standard, facilitate atomic distributed transactions across multiple, often heterogeneous, resources (e.g., databases, message queues) managed by a central Transaction Manager (TM). It utilizes a two-phase commit (2PC) protocol to ensure all resources either commit or roll back, maintaining strong consistency (ACID) across distributed components. XA specifies interfaces between a global transaction manager (TM) and resource managers (RMs). The TM coordinates the transaction's lifecycle, ensuring that all participating RMs either commit or rollback the transaction atomically.

   Key Components and Mechanisms

   1) Transaction Manager (TM): Coordinates the transaction, tracking state and deciding when to commit or rollback.
   2) Resource Manager (RM): Manages resources (like MySQL, Oracle, DB2) and provides transaction support.
   3) 2-Phase Commit (2PC):
      - Phase 1: Prepare: TM asks RMs to prepare, locking resources and preparing to commit.
      - Phase 2: Commit/Rollback: If all RMs agree, the TM commits; otherwise, it rolls back.
   4) Support: Mainstream databases like MySQL (InnoDB), Oracle, and IBM Db2 support XA protocols.



5) **Nested Transactions**: Transactions are decomposed into a hierarchy of sub-transactions, allowing parallel execution across different servers to enhance performance. Nested transactions in distributed systems structure transactions hierarchically, where a top-level transaction contains sub-transactions. Sub-transactions can commit or abort independently of their parent, providing modularity and partial failure recovery. They are crucial for concurrency control and reliable, distributed data management, enabling complex tasks to be broken into smaller, atomic units



6) **Transactional Messaging / Outbox pattern**: Ensures reliable messaging in distributed systems by atomically updating a database and publishing events. It solves the "dual-write" problem by storing messages in an outbox table within the same transaction as business data, which a separate process later publishes to a broker.

   Key Components & Flow

   a) Outbox Table: A table within the local service database that stores intended messages.
   b) Atomic Transactions: Business data update and the insertion of the message into the outbox table occur in the same ACID transaction.
   c) Message Relay/Processor: A separate service or component polls the outbox table, publishes messages to a broker (e.g., Kafka), and deletes/marks the message as processed.
   d) CDC (Change Data Capture): An alternative, more advanced approach to polling that listens to database logs to detect outbox inserts, reducing database load.

7) **Paxos / Consensus Algorithms**: Paxos is a foundational family of consensus algorithms used in distributed systems to ensure multiple nodes agree on a single value or sequence of values, even if some nodes fail. It operates using Proposers, Acceptors, and Learners to achieve consensus via majority voting (quorums), ensuring strong consistency and high availability Used in distributed systems (e.g., Google Spanner) to reach agreement on a value among nodes, ensuring fault tolerance and high consistency

8.15. Advantages of Distributed Transactions

Distributed transactions ensure atomic operations across multiple databases and services, making them critical for enterprise applications requiring strong data consistency. Below are the advantages of distributed transaction:

1) **Data Consistency (ACID):** Ensures that related data across different databases or services remain consistent. If one part of the transaction fails, the entire transaction is aborted, preventing partial updates.

2) **Fault Tolerance & Reliability:** If a participant node fails during a transaction, the distributed system can roll back changes on other nodes, ensuring the system returns to a stable state and avoiding data corruption.
3) **Guarantees Transactions:** Distributed systems guarantee the transaction. It provides features such as durability and isolation. The durability makes sure that if any transaction is committed, the changes last even if any failures occur.
4) **Atomicity Across Services:** Guarantees that complex operations involving multiple microservices are treated as a single, indivisible action.
5) **Increased Reliability:** Replicates data across nodes, meaning that if one node goes offline, the transaction management can still operate, increasing system availability.
6) **Geographic Flexibility:** Facilitates data processing near the user or source, reducing latency and allowing data to exist in multiple geographical locations while maintaining consistency.

8.16. Challenges of Distributed Transactions

The realm of distributed transactions introduces a host of challenges that are integral to distributed systems' operation. These challenges include:

1) **Concurrency Control:** Coordinating concurrent access to shared resources among multiple participants without conflicts or inconsistencies is a substantial challenge.
2) **Latency:** Distributed transactions often entail communication across networks, leading to increased latency and affecting transaction performance and response times.
3) **Fault Tolerance:** Ensuring the system's resilience to failures, such as network outages or hardware issues, is crucial to prevent data corruption or loss.
4) **Scalability:** Balancing the need for scalability with the complexities of distributed transaction management can be intricate, particularly as systems grow.
5) **Consistency:** Maintaining data consistency across distributed nodes while allowing for high availability is a delicate balancing act.
6) **Complexity:** Designing, implementing, and maintaining distributed transaction systems can be complex and require careful planning.

Addressing these challenges is fundamental to achieving robust and reliable distributed transaction management, supporting applications that depend on consistent and coordinated data operations across distributed environments.

8.17. Distributed Transactions Best Practices

When dealing with distributed systems, adopting best practices is essential to ensure the successful execution of transactions while maintaining data integrity and reliability. Here are some key recommendations:

1) **Use Proper Isolation Levels:** Choose the appropriate isolation level for your distributed transactions to balance consistency with performance.
2) **Keep Transactions Short:** Minimize the time a transaction holds locks on resources to reduce contention and enhance system concurrency.
3) **Implement Retry Mechanisms:** Design your system with automatic retry mechanisms to handle transient failures, ensuring robustness.
4) **Monitoring and Logging:** Employ comprehensive monitoring and logging to detect issues promptly, allowing for quick response and recovery.
5) **Partition Tolerance:** Design your system with partition tolerance, as network partitions can occur in distributed environments.

6) **Opt for Distributed ACID Transactions Sparingly:** Used distributed ACID Transactions judiciously, as they can introduce complexity and performance overhead.
7) **Data Partitioning:** Consider partitioning data to reduce contention and improve scalability.

By adhering to these best practices, you can navigate the challenges of distributed transactions effectively, ensuring the reliability and consistency of your distributed systems while optimizing performance and resilience

8.18. Applications of Distributed Transactions

Below are the applications of Distributed Transaction:

1) **Enterprise Resource Planning (ERP) Systems:** ERP systems consist of departments within one organization. Therefore distributed transactions are used here in order to maintain transactions from various modules such as sales, inventory, finance, and human resources management.
2) **Cloud Computing:** Distributed transactions are being used in cloud-based applications. Transactions can be done with the help of multiple data sources and ensure that data updates and operations that are performed consistently.
3) **Healthcare Systems:** Healthcare systems make use of Distributed transactions when coordinating patient records, scheduling appointments for patients, and managing the billing systems. Distributed transactions maintain data consistency and performance in healthcare systems.
4) **Financial Services:** Banking transfers, payment processing, and stock trades require atomic operations across data centres to prevent double-spending and lost updates.